

# INTRODUÇÃO AO REACT

## Sumário:

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Introdução ao React.....</b>                   | <b>2</b> |
| 1.1 Histórico do React.....                          | 2        |
| 1.2 Componentização em React.....                    | 3        |
| 1.3 Programação Declarativa com React.....           | 6        |
| 1.4 Virtual DOM (Document Object Model).....         | 7        |
| 1.5 Soluções para o desenvolvimento em React.....    | 11       |
| 1.6 Configuração do ambiente de desenvolvimento..... | 15       |
| 1.7 Criando nossa primeira aplicação React.....      | 18       |
| <b>Referências.....</b>                              | <b>1</b> |

---

## 1. Introdução ao React

React é uma biblioteca JavaScript de código aberto, desenvolvida pelo Facebook, usada para construir interfaces de usuário. O React não é um framework, e tampouco se limita ao desenvolvimento para a web. Ele é frequentemente combinado com outras bibliotecas para renderização em diferentes ambientes. Por exemplo, [React Native](#) é utilizado para a construção de aplicativos móveis nativos, enquanto [React 360](#) é usado para criar experiências de realidade virtual. Além disso, há muitas outras possibilidades, expandindo o alcance do React para diversas plataformas e contextos.

O React permite criar componentes reutilizáveis que gerenciam seu próprio estado, facilitando a construção de interfaces dinâmicas e interativas. O foco principal do React é a criação de uma experiência de usuário eficiente e performática, através do gerenciamento inteligente da atualização da interface.

Veremos a seguir mais informações sobre o seu histórico, componentização, programação declarativa, entre outros conceitos relacionados ao React.

### 1.1 Histórico do React

React foi criado pelo engenheiro de software do **Facebook**, Jordan Walke, em 2011. O desenvolvimento do React começou como uma solução interna para os problemas que o Facebook enfrentava ao construir interfaces de usuário altamente dinâmicas e interativas. O desafio era principalmente relacionado ao desempenho e à complexidade da manutenção do código.

Na época, o Facebook estava lidando com grandes volumes de dados em tempo real, especialmente em componentes como o feed de notícias. A abordagem tradicional para manipular o Document Object Model (DOM) diretamente estava se mostrando ineficiente e difícil de gerenciar à medida que as interfaces se tornavam mais complexas.

Antes de ser conhecido como React, a biblioteca foi inicialmente chamada de **FaxJS**. Jordan Walke desenvolveu essa primeira versão experimental que utilizava o conceito de um DOM virtual para otimizar as atualizações da interface de usuário. Esse conceito de **Virtual DOM** foi um dos principais diferenciais que ajudou a melhorar drasticamente a performance das aplicações web.

Em **2013**, o Facebook decidiu **liberar o React como um projeto de código aberto** durante a conferência [JSConf US](#). O lançamento público trouxe grande interesse da comunidade de desenvolvedores devido à abordagem inovadora do React para construir interfaces de usuário.

Após o lançamento, a popularidade do React cresceu rapidamente. A simplicidade e a eficiência do **paradigma declarativo**, combinadas com a **modularidade dos componentes**, fizeram com que grandes empresas e desenvolvedores individuais adotassem o React para seus projetos. O Facebook, que continua a manter e desenvolver o React, usa a biblioteca em seus produtos principais, incluindo Facebook, Instagram e WhatsApp. Outras grandes empresas como Netflix, Airbnb e Uber também adotaram o React.

## 1.2 Componentização em React

A **componentização** é um dos conceitos fundamentais em React e refere-se à prática de **dividir a interface de usuário em partes menores e reutilizáveis** chamadas componentes. Esses componentes podem ser combinados e aninhados para formar interfaces complexas de maneira modular e organizada. Cada componente é responsável por uma parte específica da interface e pode ser desenvolvido, testado e mantido de forma independente. Assim, a componentização facilita a manutenção, a reutilização de código e a escalabilidade das aplicações.

### 1.2.1 O que é um Componente?

Em React, um componente é uma **função** ou **classe JavaScript** que aceita propriedades (**props**) como entrada e retorna uma estrutura de elementos que descreve uma parte da interface de usuário. Esse retorno geralmente é feito usando JSX (uma extensão de sintaxe que se assemelha ao HTML).

Existem dois tipos principais de componentes em React:

- **Componentes Funcionais:** São componentes baseados em funções JavaScript. Eles são simples e ideais para criar componentes que apenas recebem dados e renderizam uma interface com base nesses dados. Veja um exemplo abaixo:

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

- **Componentes de Classe:** Componentes de classe são baseados em classes do JavaScript (introduzidas no ECMAScript 2015). Veja um exemplo abaixo:

```
class Greeting extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}
```

### 1.2.2 Composição de Componentes

Os componentes podem ser combinados para formar interfaces mais complexas. Um componente de nível superior pode incluir outros componentes como seus filhos, compondo uma interface completa a partir de pequenas peças modulares.

Confira, no quadro abaixo, como um componente maior pode ser composto a partir de componentes menores.

```
function App() {  
  return (  
    <div>  
      <Header />  
      <MainContent />  
      <Footer />  
    </div>  
  );  
}
```

No exemplo acima, o componente **App** é composto por três componentes menores: **Header**, **MainContent**, e **Footer**. Cada um desses componentes pode, por sua vez, ser composto por outros componentes menores.

Para entender melhor essa proposta, vamos analisar a página do campus Bento Gonçalves do Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul (IFRS):

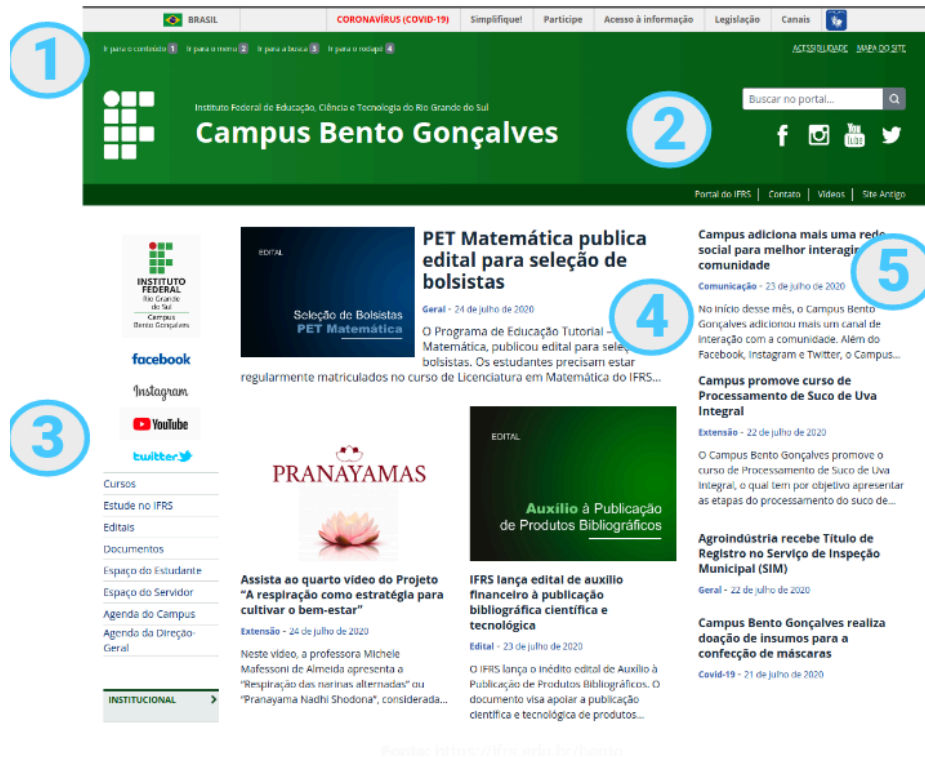


Figura 1: Exemplo de um site e seus componentes

Fonte: <https://ifrs.edu.br/bento/>

Descrição: Este site é composto por várias partes: 1) Na parte superior esquerda da página, está o logotipo da instituição e o título do site; 2) Ao lado direito do título, há ícones de redes sociais (como Facebook, X, Instagram), indicando que a instituição possui perfis nessas plataformas; 3) Na lateral esquerda da página, há um menu vertical. Este menu parece listar links para diferentes seções ou serviços oferecidos pelo campus, como "Agenda de Cursos", "Eventos", "Biblioteca", e "Contato"; 4) A área central da página é dedicada a notícias e atualizações importantes. No exemplo específico destacado, há uma notícia com o título "PET Matemática publica edital para seleção de bolsistas". 5) Na lateral direita da página, há uma seção de destaques ou anúncios especiais.

### 1.2.3 Encapsulamento

O **encapsulamento** em React refere-se à prática de isolar a lógica e o estado de um componente, de modo que ele possa funcionar de forma independente e reutilizável, sem que suas implementações internas interfiram ou sejam afetadas por outros componentes.

Veja um exemplo de encapsulamento em React no quadro abaixo:

```
function UserCard(props) {  
  return (  
    <div className="user-card">  
      <img src={props.avatarUrl} alt={` ${props.name}'s avatar`} />  
      <h2>{props.name}</h2>  
      <p>{props.bio}</p>  
    </div>  
  );  
}
```

O componente **UserCard** encapsula todos os detalhes sobre como um cartão de usuário deve ser renderizado, permitindo que seja usado de forma consistente em qualquer lugar da aplicação sem se preocupar com interferências externas.

## 1.3 Programação Declarativa com React

Os paradigmas de programação são diferentes estilos ou abordagens para escrever e organizar código. Cada paradigma oferece uma maneira distinta de pensar sobre como resolver problemas e estruturar programas. Dois dos principais paradigmas são o **imperativo** e o **declarativo**. O React utiliza predominantemente o paradigma de programação declarativa.

### 1.3.1 Programação Imperativa

A programação imperativa é um paradigma em que o desenvolvedor especifica **como** o computador deve realizar uma tarefa. Nesse estilo, você escreve uma série de instruções detalhadas que descrevem passo a passo o que o programa deve fazer para alcançar o resultado desejado. É semelhante a dar instruções precisas, como seguir uma receita de bolo, onde cada etapa deve ser executada em ordem específica. Veja no quadro abaixo um exemplo:

```
let total = 0;  
for (let i = 0; i < 10; i++) {  
  total += i;  
}  
console.log(total);
```

Neste exemplo, o código descreve explicitamente como somar os números de 0 a 9, acumulando o total em uma variável.

### 1.3.2 Programação Declarativa

A programação declarativa, por outro lado, é um paradigma onde o desenvolvedor descreve **o que** o programa deve fazer, sem especificar como fazê-lo. Em vez de dar uma sequência de passos, você define o resultado desejado, e a implementação subjacente decide a melhor forma de alcançá-lo. Assim, os detalhes de implementação, como o controle de fluxo e a manipulação de estado, são abstraídos e gerenciados pela linguagem ou framework.

No React, você descreve o que a interface de usuário deve parecer em um determinado estado, e o React se encarrega de garantir que a interface se atualize de acordo quando esse estado muda. Isso é feito através de componentes, que são funções ou classes que retornam uma descrição da interface (geralmente escrita em JSX, uma extensão de sintaxe para JavaScript que se assemelha a HTML).

Veja no quadro abaixo um exemplo de programação declarativa com React:

```
function Greeting({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in.</h1>}  
    </div>  
  );  
}
```

Neste exemplo, o componente **Greeting** declara o que deve ser exibido com base na propriedade **isLoggedIn**. Se **isLoggedIn** for **true**, o componente renderiza "**Welcome back!**", e se for **false**, ele renderiza "**Please sign in.**".

## 1.4 Virtual DOM (Document Object Model)

O Virtual DOM (Document Object Model) é uma das inovações centrais do React, projetada para otimizar a atualização de interfaces de usuário e melhorar o desempenho de aplicações web. Para entender o Virtual DOM, é útil primeiro compreender o DOM real e os desafios associados à manipulação direta do DOM.

### 1.4.1 O Que é o DOM Real?

O DOM é uma representação estruturada dos elementos de uma página web, que permite ao JavaScript interagir e manipular o conteúdo da página. Cada elemento HTML é

representado como um nó em uma árvore de objetos, e o navegador usa essa estrutura para renderizar a interface de usuário. Por exemplo, considere no quadro abaixo, o código para desenvolvimento de uma tabela HTML:

```
<table>
  <tbody>
    <tr>
      <td>Shady Grove</td>
      <td>Aeolian</td>
    </tr>
    <tr>
      <td>Over the River, Charlie</td>
      <td>Dorian</td>
    </tr>
  </tbody>
</table>
```

Neste contexto, a imagem abaixo ilustra o DOM que representa esta tabela.

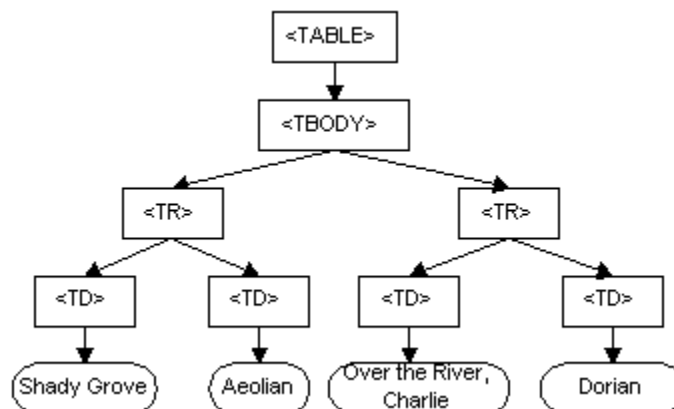


Figura 2: Representação do DOM para uma tabela HTML

Fonte: <https://www.w3.org/TR/WD-DOM-Level-2/introduction.html>

Descrição: Esta imagem apresenta um diagrama de árvore que representa a estrutura de um Documento HTML (DOM) para uma tabela. No topo da árvore, está o elemento <TABLE>, que representa uma tabela HTML. Este é o nó raiz, de onde todos os outros elementos derivam. Dentro dessa tabela (<TBODY>). O corpo da tabela contém duas linhas (<TR>). A primeira linha (<TR>) tem dois <TD>, sendo o primeiro <TD>: "Shady Grove" e o segundo <TD>: "Aeolian". A segunda linha (<TR>) também tem dois <TD>, sendo o primeiro <TD>: "Over the River, Charlie" e o segundo <TD>: "Dorian".

Manipular o DOM diretamente pode ser lento e ineficiente, especialmente em interfaces de usuário complexas que exigem muitas atualizações dinâmicas. Toda vez que o DOM real é alterado, o navegador precisa recalcular o layout, aplicar as mudanças e renderizar novamente a interface, o que pode impactar significativamente o desempenho.

### 1.4.2 O que é o Virtual DOM?

O Virtual DOM é uma representação leve e abstrata do DOM real. Em vez de interagir diretamente com o DOM real, o React mantém uma cópia virtual em memória que é muito mais rápida de manipular. O Virtual DOM é simplesmente um objeto JavaScript que espelha a estrutura do DOM real.

O funcionamento do Virtual DOM no React pode ser dividido em várias fases principais. Essas fases envolvem como o React gerencia as mudanças na interface do usuário e otimiza as atualizações ao DOM real. Aqui estão as fases detalhadas:

1. **Renderização Inicial:** Quando um componente React é renderizado pela primeira vez, o React cria um Virtual DOM que representa a estrutura da interface baseada nos componentes e estado inicial. Assim, o React converte os componentes em elementos do Virtual DOM (objetos JavaScript) que refletem a estrutura do DOM real que será renderizado pelo navegador.
2. **Detecção de Mudanças:** Quando há uma mudança no estado ou nas props de um componente, o React cria uma nova versão do Virtual DOM que reflete essas mudanças.
3. **Comparação (Diffing):** O React compara essa nova árvore do Virtual DOM com a versão anterior (um processo conhecido como *diffing*). Durante essa comparação, o React identifica exatamente quais partes da árvore do Virtual DOM foram alteradas. As mudanças podem incluir a adição, remoção ou atualização de elementos.
4. **Aplicação das Mudanças (Patching):** Após identificar as diferenças, o React aplica apenas as mudanças necessárias ao DOM real. O React minimiza as operações de manipulação do DOM real, aplicando apenas os patches (atualizações) necessários, em vez de re-renderizar toda a árvore do DOM. Isso melhora significativamente o desempenho da aplicação.
5. **Renderização (Commit):** Finalmente, as mudanças calculadas são aplicadas ao DOM real e a interface de usuário é atualizada no navegador.

A imagem abaixo ilustra o funcionamento do Virtual DOM no React:

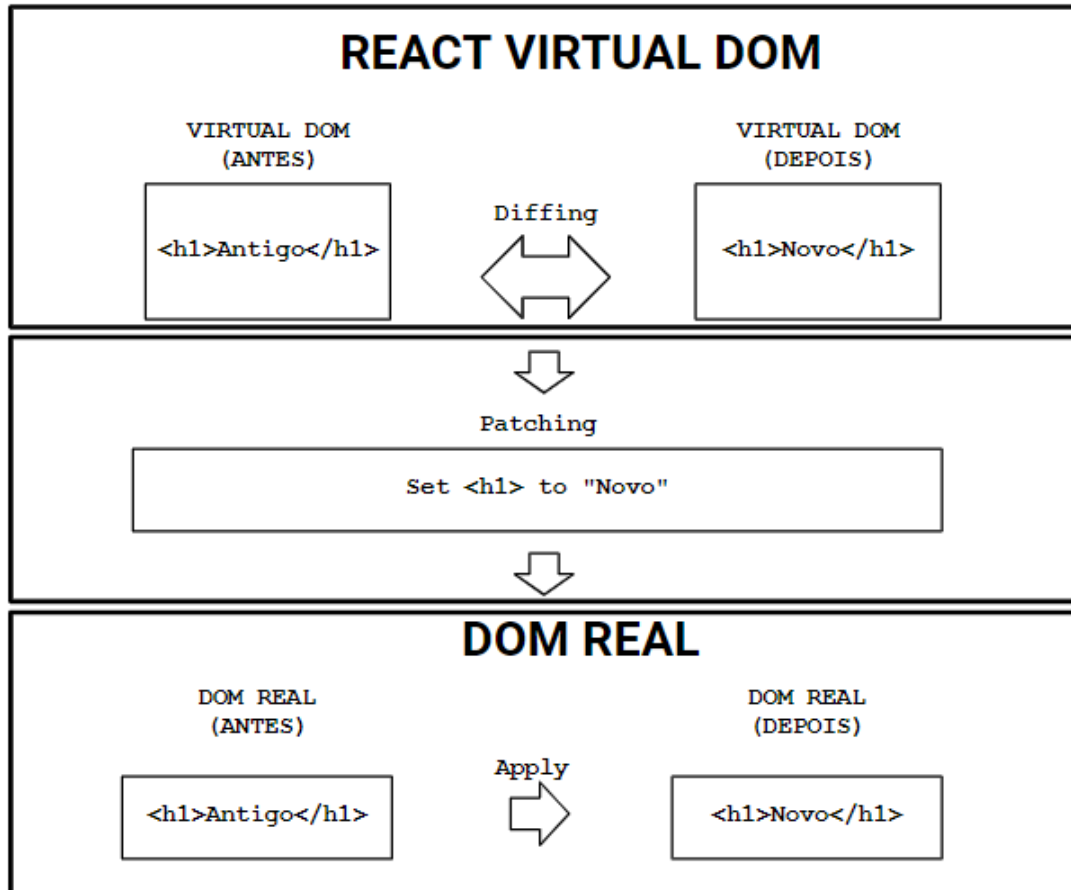


Figura 3: Representação do DOM para uma tabela HTML

Descrição: A imagem é dividida em três seções principais que ilustram as diferentes fases do processo de atualização de um elemento no DOM utilizando o Virtual DOM do React. Na primeira seção, o Virtual DOM é mostrado antes e depois de uma mudança, com o conteúdo de um elemento `<h1>` alterado de "Antigo" para "Novo", destacando o processo de comparação (diffing) realizado pelo React para identificar essa alteração. A segunda seção exibe o processo de aplicação das mudanças (patching), onde o React gera uma instrução para atualizar o elemento no DOM real, ajustando o `<h1>` para "Novo". A última seção mostra o estado do DOM real no navegador antes e depois da aplicação da atualização, com uma seta indicando que o React aplicou as mudanças necessárias ao DOM real com base na análise do Virtual DOM.

Dentre os benefícios do uso do Virtual DOM podemos destacar:

- **Performance Melhorada:** Manipular o Virtual DOM em memória é muito mais rápido do que manipular diretamente o DOM real. Como o React minimiza as interações diretas com o DOM, o desempenho geral da aplicação é significativamente melhorado.

- **Atualizações Otimizadas:** Com o Virtual DOM, o React pode garantir que apenas as partes necessárias da interface sejam atualizadas. Isso evita renderizações completas e operações de manipulação de DOM que são custosas em termos de desempenho.
- **Abstração Simples:** Desenvolvedores não precisam se preocupar com as complexidades de manipular o DOM real. Em vez disso, eles apenas declaram como a interface de usuário deve parecer para um determinado estado, e o React cuida de atualizar o DOM de maneira eficiente.

Embora o Virtual DOM ofereça grandes benefícios, ele não é uma solução mágica para todos os problemas de desempenho:

- **Complexidade Adicional:** Embora abstraia as manipulações do DOM, o Virtual DOM adiciona uma camada extra de complexidade que pode resultar em custos de processamento em situações extremas.
- **Renderizações Complexas:** Em casos muito específicos onde há muitos elementos ou atualizações muito frequentes, o processo de diffing do Virtual DOM pode se tornar uma sobrecarga.

## 1.5 Soluções para o desenvolvimento em React

A equipe do React recomenda as seguintes soluções, cada uma adequada a diferentes cenários de desenvolvimento, considerando a versão 18.3.1 do React:

- **Create React App:** Se você está iniciando seu aprendizado em React ou desenvolvendo um novo aplicativo de página única (Single Page Application - SPA), o [Create React App](#) é uma opção. Ele oferece uma configuração inicial simplificada e um ambiente de desenvolvimento otimizado para React, permitindo que você se concentre no código em vez de se preocupar com a configuração do projeto. O Create React App (CRA) utiliza o [Webpack](#) como sua ferramenta de build. Assim, quando você cria um projeto com o Create React App, ele configura automaticamente o Webpack para gerenciar a compilação e o empacotamento dos arquivos JavaScript, CSS, imagens e outros recursos do seu projeto.
- **Vite:** O [Vite](#) é uma ferramenta de compilação moderna e ágil para projetos web, incluindo aqueles desenvolvidos com React. Criada por Evan You, o criador do Vue.js, o Vite foi concebido para superar as limitações das ferramentas de build tradicionais, como o Webpack, oferecendo melhorias significativas em termos de velocidade e simplicidade.

Em vez disso, ele faz uso do [Esbuild](#). Desenvolvido em Go, o Esbuild é uma ferramenta de build altamente performática e ultrarrápida para aplicações web, focada em compilar, agrupar e minificar código JavaScript e TypeScript.

- **Next.js:** Para projetos que exigem renderização no servidor (Server-Side Rendering - SSR) em conjunto com Node.js, o [Next.js](#) é altamente recomendado. Ele facilita a criação de aplicações que necessitam de mecanismos de busca (Search Engine Optimization - SEO) aprimorado e carregamento mais rápido de páginas, além de oferecer suporte a funcionalidades avançadas como rotas dinâmicas e pré-renderização.
- **Gatsby:** O [Gatsby](#) é um framework React projetado para a criação de sites extremamente rápidos e otimizados, especialmente aqueles baseados em sistemas de gerenciamento de conteúdo (CMS). Seu principal foco é a geração de sites estáticos orientados a conteúdo, utilizando a técnica de Static Site Generation (SSG). Isso permite que os sites sejam pré-construídos em tempo de compilação, resultando em um desempenho superior e uma excelente experiência de usuário.
- **Remix:** O [Remix](#) é um framework moderno de desenvolvimento web baseado em React. Ele utiliza uma abordagem de renderização híbrida, que combina Server-Side Rendering (SSR) com Client-Side Rendering (CSR), oferecendo o melhor de ambos os mundos para criar aplicações web performáticas e dinâmicas.
- **Expo:** O [Expo](#) é um framework para o desenvolvimento de aplicativos móveis usando React Native. Ele simplifica e acelera o processo de criação de aplicativos para iOS e Android, permitindo que os desenvolvedores construam e testem seus aplicativos de forma eficiente, sem a necessidade de configuração complexa ou conhecimento profundo das plataformas nativas.

Neste curso, estaremos desenvolvendo aplicativos de página única (ou Single Page Application - SPA). SPA é um tipo de aplicação web que carrega uma única página HTML e, posteriormente, atualiza dinamicamente o conteúdo desta página à medida que o usuário interage com a aplicação. Neste contexto, precisaríamos escolher entre o **Create React App** e o **Vite**, duas ferramentas populares que utilizam empacotamento e transpilação, em nossos projetos do curso.

O **empacotamento** é o processo de combinar múltiplos arquivos JavaScript, CSS, e outros ativos em um ou mais arquivos "bundles" (pacotes). Ao criar aplicativos web, a complexidade pode aumentar rapidamente, tornando necessário dividir o código em vários arquivos para mantê-lo organizado. No entanto, carregar vários arquivos separados pode tornar o aplicativo mais lento, devido às múltiplas solicitações do navegador para buscar esses arquivos

para a sua página da web. Os empacotadores (bundlers) resolvem esse problema ao analisar as dependências do aplicativo e gerar um ou mais arquivos "bundles" (pacotes) que contém todo o código necessário, simplificando o processo e melhorando o desempenho do aplicativo.

A **transpilação** é o processo de converter código escrito em uma versão moderna de JavaScript (como ES6 ou superior) ou em outra linguagem, como TypeScript, em uma versão que seja compatível com todos os navegadores, especialmente os mais antigos.

Vejamos algumas das principais características do **Create React App** e do **Vite** a seguir.

### 1.5.1 Create React App

O Create React App (CRA) é uma ferramenta oficial desenvolvida pelo time do React para facilitar a criação de novas aplicações React. Ele oferece uma configuração de ambiente de desenvolvimento "pronta para uso" (ready-to-use), permitindo que desenvolvedores iniciem um novo projeto React rapidamente, sem se preocupar com a complexidade de configurar manualmente ferramentas auxiliares como Webpack, Babel, ESLint, etc.

O processo de compilação e transpilação é uma parte fundamental do fluxo de trabalho do Create React App. Desta forma, vamos entender como isso funciona:

- **Transpilação com Babel:** O Create React App usa [Babel](#), um transpilador JavaScript, para converter o código React escrito em JSX (uma sintaxe que mistura JavaScript e HTML) e outras funcionalidades modernas de JavaScript (como Javascript ES6) em JavaScript que é compatível com todos os navegadores. O Babel também transpila funcionalidades modernas de JavaScript, como arrow **functions**, **async/await**, e **destructuring**, para uma versão do JavaScript que é amplamente suportada pelos navegadores, garantindo que seu código funcione em ambientes mais antigos. Isso não ocorre apenas porque o navegador precisa fazer várias solicitações para buscar esses arquivos para a sua página da web, mas também porque ele pode carregar e processar códigos desnecessários desses arquivos.
- **Compilação e Empacotamento com Webpack:** O Create React App usa o [Webpack](#) como bundler, uma ferramenta que empacota todos os módulos JavaScript do projeto, além de outros ativos como CSS, imagens e fontes, em um ou mais arquivos "bundles". Assim, o Webpack analisa as dependências do seu código JavaScript (importações e exportações) e cria um gráfico de

dependências. Ele então empacota todos os módulos em um único arquivo ou em vários arquivos otimizados para serem carregados pelo navegador. Durante o desenvolvimento, Webpack usa Hot Module Replacement (HMR) para permitir a atualização de módulos no navegador sem recarregar a página. Durante o desenvolvimento, o Create React App inicia um servidor de desenvolvimento. O Webpack observa as alterações no código e recompila os arquivos conforme necessário.

Neste contexto, o Create React App simplifica significativamente o processo de configuração e execução de um ambiente de desenvolvimento React. Ele abstrai as complexidades de transpilação com Babel e empacotamento com Webpack, permitindo que os desenvolvedores se concentrem no código do aplicativo.

Entretanto, o Webpack é configurado para compilar todo o código antecipadamente, criando pacotes de recursos que são usados pelo navegador para renderizar o aplicativo. Isso significa que, durante o desenvolvimento, cada vez que o código é modificado, o Webpack deve recompilar todos os pacotes, o que pode levar a tempos de compilação mais longos e um processo de desenvolvimento mais lento.

### 1.5.2 Vite

Vite é uma ferramenta moderna de build para front-end que foi desenvolvida por Evan You, o criador do Vue.js. O Vite é projetado para ser um substituto mais moderno para ferramentas como Webpack, especialmente para projetos que utilizam frameworks como React, Vue, Svelte, e outros. Ele se destaca pela sua abordagem inovadora para o desenvolvimento e build de aplicações.

Vamos entender como funciona o processo de compilação e transpilação com o Vite:

- **Transpilação com ESBuild:** O [Esbuild](#) é uma ferramenta extremamente rápida, escrita em Go, que transpila código moderno de JavaScript (ES6 ou superior) e JSX (uma sintaxe que mistura JavaScript e HTML) em JavaScript compatível com os navegadores. Cabe salientar que o ESBuild é notavelmente mais rápido que transpiladores tradicionais como Babel.
- **Compilação e Empacotamento com Rollup:** Vite utiliza uma abordagem híbrida para empacotamento. Durante o desenvolvimento, ele serve os módulos JavaScript diretamente ao navegador como ES Modules (ESM). Essa abordagem

torna a compilação muito mais rápida, porque apenas os módulos que são necessários para renderizar a página atual são compilados, e não todo o aplicativo, resultando em tempos de inicialização quase instantâneos. Para produção, o Vite usa [Rollup](#), um bundler eficiente que realiza empacotamento, minificação, e outras otimizações.

Observamos que, ao contrário das ferramentas tradicionais que processam todo o código do projeto antes de iniciar o servidor de desenvolvimento, o Vite adota uma estratégia de Hot Module Replacement (HMR) extremamente eficiente. Ele atualiza apenas os módulos que realmente mudaram, sem precisar recarregar a página inteira. Isso significa que, durante o desenvolvimento, cada vez que um módulo é modificado, apenas esse módulo precisa ser recompilado e atualizado no navegador, em vez de recompilar todo o aplicativo. Esse processo resulta em tempos de compilação significativamente mais rápidos, proporcionando uma experiência de desenvolvimento mais ágil e responsiva. Essa abordagem é viabilizada pelo uso do recurso nativo do navegador conhecido como ESM (ECMAScript Modules) para a importação e exportação de módulos Javascript.

Neste curso, desta forma, optamos por utilizar o Vite em nossos projetos.

## 1.6 Configuração do ambiente de desenvolvimento

A configuração de um ambiente de desenvolvimento em React envolve preparar o sistema para que você possa desenvolver, testar e depurar aplicações React de forma eficiente. Aqui estão os principais componentes e passos necessários para configurar esse ambiente:

### 1.6.1 Instalação do Node.js e npm

O Node.js é uma plataforma de runtime baseada em JavaScript que permite a execução de código JavaScript fora do navegador, geralmente no lado do servidor. Ele foi construído sobre o motor V8 do Google Chrome, que compila JavaScript diretamente em código de máquina, tornando-o muito rápido e eficiente.

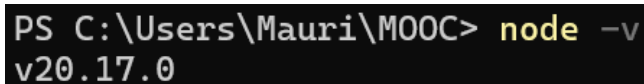
Instalar o **Node.js** é essencial para desenvolver com React por várias razões, principalmente relacionadas ao gerenciamento de dependências, execução de scripts de desenvolvimento, e ferramentas de build. Quando você instala o [Node.js](#), ele vem automaticamente com o **npm (Node Package Manager)**, que é o gerenciador de pacotes mais utilizado no ecossistema JavaScript. O npm permite que você instale, atualize e gerencie bibliotecas e ferramentas necessárias para o desenvolvimento em React.

**OBS:** Alternativamente, você pode usar o **Yarn**, outro gerenciador de pacotes que também depende do Node.js. O [Yarn](#) oferece funcionalidades semelhantes ao npm e é conhecido por sua velocidade e confiabilidade.

Desta forma, baixe e instale o Node.js (que inclui o npm) a partir do [site oficial](#). Em seguida, verifique se o Node.js está instalado corretamente. Para isso, abra o terminal (no macOS ou Linux) ou o Prompt de Comando/PowerShell (no Windows) e digite o seguinte comando:

```
node -v
```

Este comando exibe a versão do Node.js instalada. Se o Node.js estiver instalado corretamente, você verá algo como ilustrado na figura abaixo:



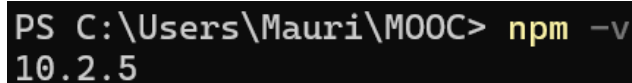
```
PS C:\Users\Mauri\M00C> node -v  
v20.17.0
```

Figura 4: O número da versão do Node.js instalado em seu computador

Descrição: A imagem é uma captura de tela de um terminal ou prompt de comando. O comando `node -v` foi executado corretamente, e o sistema retornou a versão "v20.17.0" do Node.js, indicando que o Node.js está instalado e funcionando corretamente no computador do usuário.

Nesta imagem, podemos verificar o número da versão do Node.js instalado no meu computador, neste caso, a versão "20.17.0". Cabe ressaltar que este número pode variar dependendo da versão que você instalou.

Para verificar se o npm foi instalado corretamente, digite o seguinte comando no terminal ou prompt de comando:



```
PS C:\Users\Mauri\M00C> npm -v  
10.2.5
```

Figura 5: O número da versão do npm instalado em seu computador

Descrição: A imagem é uma captura de tela de um terminal ou prompt de comando no Windows e mostra o resultado de um comando digitado para verificar a versão do npm instalada no sistema. O comando `npm -v` foi executado corretamente, e o sistema retornou a versão "10.2.5" do npm.

Nesta imagem, podemos verificar o número da versão do npm instalado em meu computador, neste caso a versão "10.2.5".

---

## 1.6.2 Instalação de um Editor de Código

Para programação com React, existem vários softwares e ferramentas que podem melhorar sua produtividade e facilitar o desenvolvimento. Recomendamos o Visual Studio Code (VS Code), que é um editor de código-fonte gratuito, leve e altamente extensível, desenvolvido pela Microsoft. É uma ferramenta multiplataforma que funciona em Windows, macOS e Linux. O [VS Code](#) é amplamente utilizado por desenvolvedores devido à sua interface amigável, suporte a várias linguagens de programação e grande quantidade de extensões disponíveis, que permitem personalizar e expandir sua funcionalidade para atender às necessidades específicas de desenvolvimento.

O VS Code é um dos editores de código mais populares e é recomendado para o desenvolvimento em React. Ele possui uma gama de extensões que facilitam o desenvolvimento, como suporte ao JSX, ESLint, Prettier, etc. Alguns dos principais recursos do VS Code incluem:

- **Integração de Controle de Versão:** Suporte integrado para Git, permitindo que os desenvolvedores gerenciem facilmente o controle de versão diretamente do editor.
- **Depuração Integrada:** Ferramentas de depuração para várias linguagens de programação, incluindo JavaScript.
- **Extensões:** Um vasto marketplace de extensões que permite adicionar suporte a linguagens, temas, depuração adicional, integração de serviços, e muito mais.
- **Terminal Integrado:** Um terminal integrado que permite executar comandos sem sair do editor.
- **IntelliSense:** Autocompletar código com sugestões inteligentes, com base no contexto do código e nas definições de tipos.
- **Git e Controle de Versão:** Com o suporte integrado ao Git, o VS Code facilita o gerenciamento de repositórios de código diretamente do editor.

O VS Code possui várias extensões que são extremamente úteis para o desenvolvimento em React. Algumas das mais populares incluem:

- **ESLint:** Ferramenta para identificar e corrigir problemas no código JavaScript.
- **Prettier:** Um formatador de código que ajuda a manter a consistência na formatação do código.
- **React Developer Tools:** Extensão que permite inspecionar e depurar componentes React diretamente no navegador.

Os ambientes de desenvolvimento integrado (IDEs) como [Visual Studio](#) e [WebStorm](#) oferecem uma experiência mais robusta com ferramentas integradas para desenvolvimento web. Entretanto, não são necessários para o conteúdo que vamos abordar neste curso.

Uma outra alternativa, útil para testar pequenas partes de código, seria utilizar algum dos editores de código-fonte online. Estas são ferramentas baseadas na web que permitem aos desenvolvedores escrever, editar e testar código diretamente no navegador. Eles oferecem várias funcionalidades que facilitam o desenvolvimento de software sem a necessidade de instalar um ambiente de desenvolvimento local. Podemos citar os seguintes editores de código-fonte online: [CodeSandbox](#), [StackBlitz](#), [CodePen](#), [JSFiddle](#) e [Replit](#).

### 1.6.3 Instalação de navegadores modernos

Para testar e visualizar seu trabalho, você precisará de navegadores web modernos. Aqui estão algumas sugestões para diferentes sistemas operacionais:

- **Linux:** [Firefox](#), [Chrome](#), [Opera](#), [Brave](#).
- **Windows:** [Firefox](#), [Chrome](#), [Opera](#), [Brave](#), [Microsoft Edge](#). (O Edge já vem instalado por padrão no Windows 10; para versões anteriores, como o Windows 7, recomendamos instalar o Internet Explorer 11 ou outro navegador moderno.)
- **Mac:** [Firefox](#), [Chrome](#), [Opera](#), [Brave](#), [Safari](#). (O Safari já vem instalado por padrão no macOS e iOS.)

## 1.7 Criando nossa primeira aplicação React

Agora, você está pronto para criar sua primeira aplicação React. Neste passo, você usará o npm, o gerenciador de pacotes do Node.js, para executar um script remoto que configurará automaticamente seu projeto. Esse script copiará todos os arquivos necessários para um novo diretório e instalará as dependências necessárias.

### 1.7.1 Criando o projeto React com o Vite

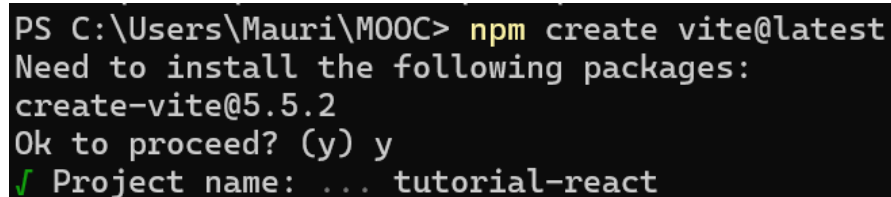
Vamos começar criando um novo projeto utilizando o Vite, essa ferramenta de build moderna para desenvolvimento front-end. Para isso, digite o seguinte comando no terminal:

```
npm create vite@latest
```

No exemplo acima, **vite@latest** é o pacote que o npm deve buscar e executar. O **@latest** assegura que você está usando a versão mais atual do Vite disponível no **repositório npm**.

Quando você executa **npm create vite@latest**, o **npm** faz o download e executa o pacote **create-vite**, que é responsável por configurar um novo projeto Vite. O pacote **create-vite** contém um **script que define como a estrutura do projeto será criada**, quais arquivos serão incluídos, e como as dependências serão instaladas.

Durante a execução, o Vite pode solicitar que você escolha um **nome para o projeto**. Informe o seguinte nome **"tutorial-react"**, conforme figura abaixo.

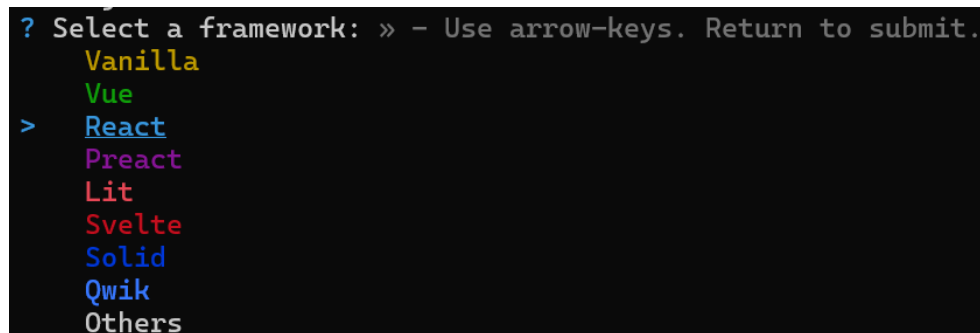


```
PS C:\Users\Mauri\MOOC> npm create vite@latest
Need to install the following packages:
create-vite@5.5.2
Ok to proceed? (y) y
✓ Project name: ... tutorial-react
```

Figura 6: Criando um novo projeto React utilizando o Vite

Descrição: A imagem mostra um terminal ou prompt de comando no Windows onde o usuário está criando um novo projeto React utilizando o Vite. O comando **npm create vite@latest** foi executado, e o sistema indicou que precisava instalar o pacote **create-vite** na versão 5.5.2. O usuário confirmou a instalação digitando "y" e, em seguida, foi solicitado a fornecer um nome para o projeto. O usuário escolheu "tutorial-react" como o nome do projeto, e o processo de criação do projeto está em andamento.

Em seguida, o Vite pode solicitar que você escolha um **framework/biblioteca** para o projeto. Por exemplo, você pode escolher entre **frameworks/bibliotecas** como React, Vue, Svelte, um projeto Vanilla (JavaScript puro), entre outros. Neste caso, escolha a opção **React**, tal como mostra a figura abaixo:



```
? Select a framework: » - Use arrow-keys. Return to submit.
  Vanilla
  Vue
>  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```

Figura 7: Escolher um framework para um novo projeto React usando o Vite

Descrição: A imagem mostra um terminal ou prompt de comando no Windows onde o usuário é solicitado a escolher um framework para um novo projeto. A opção "React" está atualmente selecionada.

Em seguida, o Vite pode solicitar que você escolha qual **linguagem de programação** você irá utilizar para o projeto, tais como: JavaScript e TypeScript. Vamos selecionar a opção **JavaScript + SWC**, conforme figura abaixo:

```
? Select a variant: » - Use arrow-keys. Return to submit.
  TypeScript
  TypeScript + SWC
  JavaScript
> JavaScript + SWC
  Remix ↗
```

Figura 8: Escolher uma linguagem para seu projeto React com o Vite

Descrição: A imagem mostra um terminal ou prompt de comando no Windows onde o usuário é solicitado a escolher uma variante (linguagem) para seu projeto React. No momento, a opção "JavaScript + SWC" está selecionada.

A opção "JavaScript + SWC" refere-se ao uso do compilador [SWC](#) (Speedy Web Compiler), desenvolvido em Rust, usado para transpilar e otimizar o código JavaScript e TypeScript. O SWC é uma alternativa moderna ao Babel, conhecida por ser extremamente rápida e eficiente.

Este processo de configuração inicial do projeto leva alguns segundos. Ao terminar, o Vite repassa instruções para que você termine de instalar as dependências do seu projeto. Desta forma, digite o seguinte comando no terminal para **entrar no diretório** recém criado do nosso projeto, conforme quadro abaixo:

```
cd tutorial-react
```

Em seguida, **instale as dependências** necessárias do projeto executando no terminal o seguinte comando:

```
npm install
```

Até aqui, você criou um projeto React usando o Vite e adicionou todas as dependências ao projeto. Você pode, agora, **abrir a aplicação no editor Visual Studio Code**. Para isso, dentro do diretório tutorial-react digite o seguinte comando no terminal:

```
code .
```

Após a execução deste comando, o Visual Studio Code deverá abrir com a pasta raiz do seu projeto sendo acessada.

Quando você cria um projeto React usando o Vite, ele gera uma estrutura de diretórios organizada e otimizada para facilitar o desenvolvimento. Vamos acompanhar na figura abaixo a **estrutura típica** (ou sua variação) de um **projeto React criado com o Vite**:

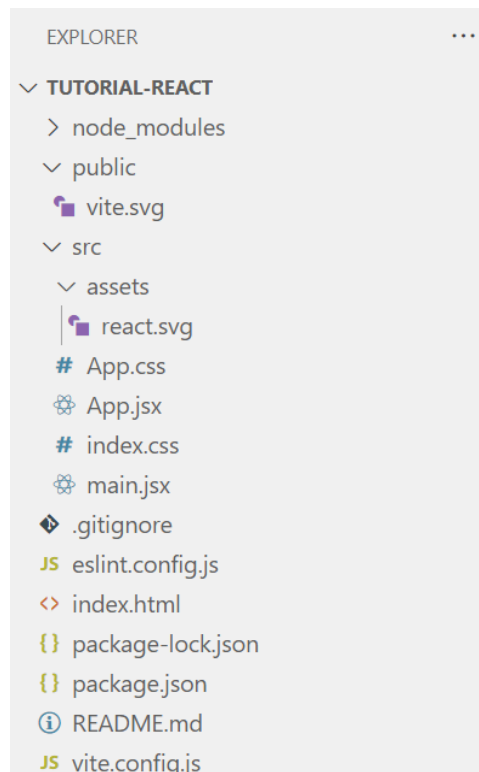


Figura 9: Estrutura de diretórios e arquivos de um projeto React criado com o Vite

Descrição: A imagem mostra a estrutura de diretórios e arquivos de um projeto React chamado "tutorial-react" no Visual Studio Code. Essa estrutura mostra a organização típica de um projeto React criado com Vite, incluindo arquivos de configuração, dependências e o código-fonte principal.

Conforme observado na imagem acima, temos a seguinte estrutura de diretórios e arquivos em nosso projeto:

- **node\_modules/**: Este diretório é gerado automaticamente pelo npm e contém todas as dependências instaladas para o projeto. Essas dependências incluem bibliotecas e pacotes que o projeto utiliza, como React, ReactDOM, e o próprio Vite.
- **public/**: A pasta **public** é usada para armazenar arquivos estáticos que não precisam ser processados pelo Vite. O conteúdo dessa pasta é servido diretamente como está, sem qualquer modificação. Imagens, fontes, etc, são exemplos de arquivos estáticos que você deseja acessar diretamente.

- **src/:** A pasta **src** contém todo o código-fonte do projeto. Aqui é onde você escreverá seus componentes React, definirá seus estilos e gerenciará a lógica da aplicação. Veja abaixo seu conteúdo típico:
  - **assets/:** Diretório usado para armazenar recursos como imagens, fontes, etc., que são utilizados pelos componentes da aplicação.
  - **App.jsx:** Este é o componente principal da aplicação. É comum que o **App.jsx** seja o ponto central onde outros componentes são montados e gerenciados.
  - **App.css:** Arquivo de estilos associado ao componente **App.jsx**. Ele contém as regras CSS específicas para o App.
  - **main.jsx:** O ponto de entrada da aplicação React. Este arquivo é responsável por renderizar o componente App no DOM. Ele importa o **App.jsx** e utiliza ReactDOM.render para montá-lo em um elemento HTML com um determinado **id** (geralmente **root**).
  - **index.css:** Arquivo de estilos globais. Ele é geralmente importado no **main.jsx** para aplicar estilos que afetam toda a aplicação.
- **gitignore:** Este arquivo lista os arquivos e diretórios que devem ser ignorados pelo [Git](#), um sistema de controle de versão de código-fonte. Normalmente, inclui a pasta node\_modules/, os arquivos de build, e outras configurações locais que não precisam ser rastreadas pelo controle de versão.
- **eslint.config.js:** Este é um arquivo de configuração para o [ESLint](#), uma ferramenta de análise estática (linter) usado para identificar e corrigir problemas no código JavaScript, incluindo código JSX de projetos React.
- **index.html:** Este é o **arquivo HTML principal** que o Vite utiliza para injetar o JavaScript gerado. Ele contém um **<div>** com um **id**, como **root**, onde o React renderizará sua aplicação.
- **package-lock.json:** esse arquivo é gerado automaticamente pelo **npm** e registra as versões exatas de todas as dependências do seu projeto. Ele garante que todos os desenvolvedores de um projeto tenham exatamente as mesmas versões de dependências instaladas em seus computadores;
- **package.json:** Contém metadados sobre o projeto, como o nome, versão, scripts, dependências de produção e de desenvolvimento.
- **README.md.** Arquivo Markdown que normalmente contém informações sobre o projeto, como instruções de instalação, uso, e notas importantes. É um lugar ideal para documentar detalhes específicos do projeto.

- **vite.config.js.** Este arquivo contém a configuração do Vite. Por padrão, ele pode estar bastante simples ou até mesmo vazio, mas à medida que o projeto cresce, você pode configurar aspectos como aliás de caminhos, proxies de API, plugins, entre outros.

## 1.7.2 Iniciando o servidor local

Neste passo, você inicializará um servidor local e executará o projeto em seu navegador. Digite a seguinte linha de comando no terminal:

```
npm run dev
```

Ao executar esse script, você iniciará um servidor local de desenvolvimento, executará o código do projeto, iniciará um observador que detecta alterações no código e abrirá o projeto em um navegador web. Ele irá rodar a aplicação em modo desenvolvimento em <http://localhost:5173/>. No navegador, você verá o seguinte projeto modelo do React:

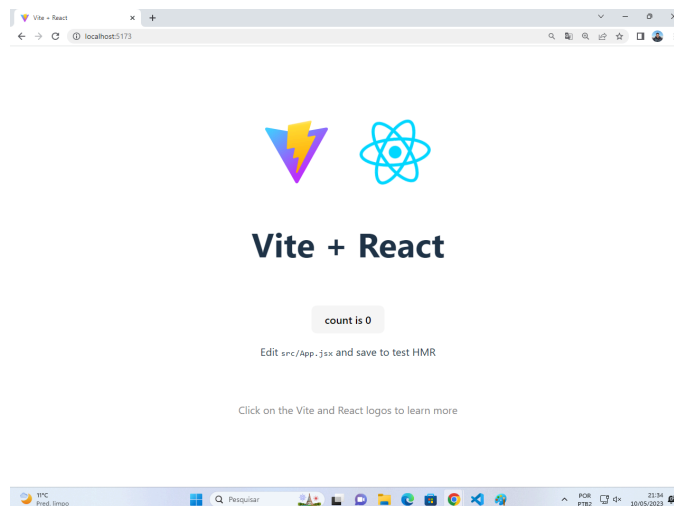


Figura 10: Página inicial de um projeto React criado com o Vite executada em um navegador web

Descrição: A imagem mostra uma janela do navegador exibindo a página inicial de um projeto React criado com o Vite. No topo da página, há os logos do Vite (um raio) e do React (um átomo estilizado). Abaixo dos logos, há o texto "Vite + React" seguido de um contador que inicialmente exibe "count is 0". Abaixo disso, uma mensagem instrui o usuário a editar o arquivo src/App.jsx e salvar para testar o HMR (Hot Module Replacement), que permite ver as mudanças em tempo real sem recarregar a página.

Até agora, você iniciou o servidor de desenvolvimento e está executando seu primeiro código React com sucesso.

**OBS:** Enquanto o script estiver em execução, você terá um servidor local ativo. Para parar o script, digite **CTRL+C** na guia do terminal onde seu script estiver em execução.

---

## Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- DEVLIN BASILAN DULDULAO, D. B.; CABAGNOT, R. J. L. Practical Enterprise React: Become an Effective React Developer in Your Team. USA: APRESS, 2021.
- ELROM, E. React and Libraries: Your Complete Guide to the React Ecosystem. USA: APRESS, 2021.
- MDN WEB DOCS. Getting started with React. Disponível em: [https://developer.mozilla.org/en-US/docs/Learn/Tools\\_and\\_testing/Client-side\\_JavaScript\\_frameworks/React\\_getting\\_started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started). Acessado em 21 jun. 2024.
- REACT DOCS. Disponível em <https://pt-br.react.dev/>. Acessado em 15 jun. 2024.
- ROLDÁN, C. S. React 17: Design Patterns and Best Practices. UK: Packt Publishing, 2021.
- SCOTT, A. D. JavaScript Everywhere. USA: O'Reilly Media, 2020.
- UZAYR, S. B.; CLOUD, N.; AMBLER, T. JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now. USA: APRESS, 2019.
- W3SCHOOLS. React Tutorial. Disponível em: <https://www.w3schools.com/react>. Acessado em 17 jun. 2024.